Computer Science Competition
District 2026
Programming Problem Set

I. General Notes

1. Do the problems in any order you like. They do not have to be done in order from 1 to 12.

2. All problems have a value of 60 points.

3. There is no extraneous input. All input is exactly as specified in the problem. Unless specified by the problem, integer inputs will not have leading zeros. Unless otherwise specified, your program should read to the end of file.

4. Your program should not print extraneous output. Follow the form exactly as given in the problem.

5. A penalty of 5 points will be assessed each time that an incorrect solution is submitted. This penalty will only be assessed if a solution is ultimately judged as correct.

II. Names of Problems

Number	Name
Problem 1	Dax
Problem 2	Newton
Problem 3	Charlie
Problem 4	Rumi
Problem 5	Astra
Problem 6	David
Problem 7	Tesla
Problem 8	Bohr
Problem 9	Tristan
Problem 10	Monoco
Problem 11	Esquie
Problem 12	Sophie

1. Dax

Program Name: Dax.java **Input File: none**

Dax has enjoyed an amazing spring break and can't lose the joy felt for the awesomeness that is the month of March. As such, Dax has designed an awesomely ASCII title for the calendar being put together by the class.

Input:
None

Output:
Output the title March as shown and formatted below. There are no blank spaces added at the end of each line. The last 5 lines all begin with zero spaces in front of the first character.

Sample output:

```
                                    888
                                    888
                                    888
88888b.d88b.  8888b. 888d888 .d8888b88888b.
888 "888 "88b    "88b888P"  d88P"   888 "88b
888  888  888.d888888888    888     888  888
888  888  888888  888888    Y88b.   888  888
888  888  888"Y888888888     "Y8888P888  888
```

2. Newton

Program Name: Newton.java **Input File: newton.dat**

In history class you've begun studying the pyramids. Rather fascinated by their structural integrity, you decide to write a program that will output pyramids of different sizes.

Input:
The first input line will contain a number N (1 <= N <= 100) denoting the number of test cases that follow. Each test case will contain an odd integer H (1 <= H <= 1000) denoting the height of the pyramid (number of layers).
Output:
The resulting pyramid with height of H for all given values of H, with an empty line between pyramids. A brick of the pyramid should be denoted by "*".

Sample input:

```
3
1
3
5
```

Sample output:

```
*

  *
 ***
*****

    *
   ***
  *****
 *******
*********
```

3. Charlie

Program Name: Charlie.java Input File: charlie.dat

Charlie has recently started doing ciphers. He wants to practice rotating strings so he can start doing Caesar ciphers. You will be given a string, and a number of positions to rotate (positive is right, negative is left).

Input:
The input will begin with one integer, `n` (`0 < n <= 1000`). Each test case will consist of one string `s`, followed by a space, followed by an integer `m` (`-1000 <= m <= 1000`). You will need to rotate the string `s` `m` spaces.

Output:
For each test case, output the string after the rotation has occurred.

Sample input:
```
3
ABCDEFGH 3
ABCDEFGH 8
ABCDEFGH -1
```

Sample output:
```
FGHABCDE
ABCDEFGH
BCDEFGHA
```

4. Rumi

Program Name: Rumi.java **Input File: rumi.dat**

Rumi is programming a surveillance drone to navigate a 3D coordinate space. The drone must visit a series of waypoints in order after leaving its starting point (3,4,0). To complete the mission, the drone must return to its starting point (3, 4, 0) after visiting the last waypoint. All waypoints are constrained within a 15 x 15 x 15 cubic area. Calculate the total distance traveled to 2 decimal places. The 3D distance formula might prove useful: $\sqrt{(x2-x1)^2+(y2-y1)^2+(z2-z1)^2}$

Input:
The data file will begin with a single integer **N** on the first line. The following **N** lines will each contain 3 doubles, separated by spaces, representing x, y, and z. 0 <= x,y,z <= 15

Output:
A single line: `Total Distance: VALUE units`
The value should be displayed to 2 decimal places..

Sample input:
```
2
3.0 4.0 5.0
0.0 0.0 5.0
```

Sample output:
```
Total Distance: 17.07 units
```

5. Astra

Program Name: Astra.java **Input File: astra.dat**

Astra is chair of the prom committee for the student council. The council this year is trying out group orders to encourage more students to attend with their friends. There are two (2) levels of tickets available, REGULAR and VIP. Also, prices are different based on the student's grade level. If the group order is for five (5) or more tickets, the group gets a 10% discount on each ticket in the group. Ticket prices are in the table below:

Grade Level	Ticket Type	Base Price
Senior (12)	REGULAR	$40.00
Senior (12)	VIP	$55.00
Junior (11)	REGULAR	$45.00
Junior (11)	VIP	$60.00
Soph/Fresh (9-10)	REGULAR	$50.00
Soph/Fresh (9-10)	VIP	$65.00

Input:
The input file is broken into groups:
The first line is the group leader's name (one word)
The next line is a single integer **N**, representing the number of students in the groups.
The next **N** lines contain the ticket data for each student in the format: an int (grade level), a space, and `VIP/REGULAR` (ticket type)

Output:
Each group ticket order will be displayed as follows:
The first line will read "`GROUP ORDER: `" followed by the name on the order. (minus the quotes)
The second line will consist of 42 dashes (−).
The next **N** lines will read "`Student #X (grade Y, VIP/REGULAR)`", then, beginning under the 34th dash, "`: $ `", followed by the cost of the ticket to 2 decimal places with the decimal point aligned under the 41st dash.
After the student tickets are all listed, output a line of 42 dashes (−).
The next line of output begins with "`Group Total`". If the group qualifies for a discount, follow this with a space and "`(Disc. Applied)`". The line concludes with "`: $ `" beginning under the 34th dash followed by the total group amount to 2 decimal places with the decimal point aligned under the 41st dash.
End each group with two (2) blank lines.

Sample input:

```
Smith
3
12 VIP
11 REGULAR
12 REGULAR
Johnson
5
10 VIP
9 REGULAR
10 VIP
9 REGULAR
10 VIP
```

Sample output:

```
GROUP ORDER: Smith
------------------------------------------
Student #1 (Grade 12, VIP)       : $  55.00
Student #2 (Grade 11, REGULAR)   : $  45.00
Student #3 (Grade 12, REGULAR)   : $  40.00
------------------------------------------
Group Total                      : $ 140.00


GROUP ORDER: Johnson
------------------------------------------
Student #1 (Grade 10, VIP)       : $  58.50
Student #2 (Grade 9, REGULAR)    : $  45.00
Student #3 (Grade 10, VIP)       : $  58.50
Student #4 (Grade 9, REGULAR)    : $  45.00
Student #5 (Grade 10, VIP)       : $  58.50
------------------------------------------
Group Total (Disc. Applied)      : $ 265.50
```

6. David

Program Name: David.java Input File: david.dat

David is the newest member of a club called the "Rome did it better" club. He is being ostracized from the rest of the club due to his inability to understand roman numerals, and he needs your help to create a program that can convert any roman numeral into a value he can understand. The following roman numerals correspond to these values:

I	1	V	5	X	10
L	50	C	100	D	500
M	1000				

Input:
The input will begin with one integer, `n` (`0 < n <= 1000`). Each test case will consist of one line, containing a string denoting the roman numeral value given.

Output:
For each test case, output the integer value of the given roman numeral string. This value will be between 0 and 3999.

Sample input:
```
3
ILV
MCMXCIX
CCCLXV
```

Sample output:
```
54
1999
365
```

7. Tesla

Program Name: Tesla.java **Input File: tesla.dat**

You've been hired to write an inventory management system for a small warehouse. Each item belongs to exactly one category. The system needs to process commands to manage items and answer queries about stock levels.

Input:
The first line contains a single integer **N** ($1 \leq N \leq 1000$), the number of commands to process.
The next N lines each contain a command in one of the following formats:

- ADD category item quantity — Add the specified quantity of the item to inventory under the given category. If the item already exists, add to its quantity (the category remains unchanged from the first ADD).
- REMOVE item quantity — Remove the specified quantity of the item. If not enough stock, reduce to 0. If the item doesn't exist, ignore the command.
- QUERY item — Output the current quantity of the specified item (0 if not found).
- CATEGORY cat — Output the total quantity of all items in that category, then list all items in that category with quantity > 0 in alphabetical order as item quantity. If the category has no items in stock, output 0 followed by EMPTY.
- TOTAL — Output the number of distinct items currently in stock (quantity > 0).
- LIST — Output all items with quantity > 0 sorted alphabetically by category, then by item name, in the format category item quantity. Output EMPTY if nothing is in stock.

Constraints

- Item names and category names consist of lowercase letters only, at most 20 characters each.
- All item names are unique across all categories.
- Quantities are positive integers not exceeding 10,000.
- Each item belongs to exactly one category (determined by its first ADD command).
- There will be at least one QUERY, CATEGORY, TOTAL, or LIST command.

Output:
For each QUERY command, output a single integer on its own line.
For each CATEGORY command, output the total quantity on one line, followed by the item listing (or EMPTY) as described.
For each TOTAL command, output a single integer on its own line.
For each LIST command, output the items as specified (or EMPTY).

Sample input:

```
14
ADD fruit apples 50
ADD fruit bananas 30
ADD dairy milk 20
ADD fruit apples 25
QUERY apples
CATEGORY fruit
ADD dairy cheese 15
TOTAL
REMOVE milk 20
CATEGORY dairy
LIST
REMOVE bananas 30
CATEGORY fruit
TOTAL
```

Sample output:

```
75
105
apples 75
bananas 30
4
15
cheese 15
dairy cheese 15
fruit apples 75
fruit bananas 30
75
apples 75
2
```

8. Bohr

Program Name: Bohr.java **Input File: bohr.dat**

Two teams, Red (R) and Blue (B), are battling for control of strategic points on a battlefield. Each team has soldiers that spawn from team bases and move toward objectives. When soldiers from opposing teams occupy the same cell, they engage in combat. Fallen soldiers respawn at their team's bases and rejoin the battle. Teams earn points by holding control points — at the start of each day, teams receive points equal to the value of each control point they currently hold. Your task is to simulate T days of battle and report the final scores.

Game Rules
Each day, process the following phases in order:
Phase 0: Scoring
Each team earns points equal to the sum of the values of all control points they currently control.
Phase 1: Target Selection
Each soldier selects a target cell:

1. If there exists any control point **not controlled by your team** (enemy-controlled or neutral): target the **closest** such control point
2. Otherwise (your team controls all points): target the **closest enemy spawn base**

Distance is calculated using Euclidean distance from the soldier's current position.
Tie-breakers (apply in order):

- Prefer higher point value (for control points; spawns have value 0)
- Prefer smaller row index
- Prefer smaller column index

Phase 2: Movement
Each soldier moves **one step** toward their target. Soldiers prioritize moving vertically (up or down) before moving horizontally (left or right). Soldiers cannot move through walls. If a soldier cannot make progress toward their target, they remain in place. Multiple soldiers may occupy the same cell.
Phase 3: Combat
For each cell containing soldiers from **both** teams (processing cells in reading order: top-to-bottom, left-to-right), resolve combat:

1. Generate a random number (0-99) for each Red soldier using the Red combat seed
2. Generate a random number (0-99) for each Blue soldier using the Blue combat seed
3. Pair the highest-rolling Red soldier against the highest-rolling Blue soldier, the second-highest Red against the second-highest Blue, and so on
4. For each pair: the higher roll wins and the losing soldier is eliminated. If tied, **both** soldiers are eliminated
5. Unpaired soldiers (when teams have unequal numbers) survive without fighting

Eliminated soldiers are removed and will respawn next phase.
Phase 4: Capture
For each control point:

- If soldiers from **exactly one team** are present, that team **captures** the point
- Otherwise, ownership is unchanged (no soldiers, or soldiers from both teams still present)

Control points start as neutral (uncontrolled).
Phase 5: Respawn
Each eliminated soldier respawns at a randomly selected spawn base belonging to their team. Red soldiers use the Red respawn seed; Blue soldiers use the Blue respawn seed.

Input:
The first line contains three integers N M T representing the grid rows, grid columns, and number of days. The second line contains two integers RC RR representing the Red combat seed and Red respawn seed. The third line contains two integers BC BR representing the Blue combat seed and Blue respawn seed. The fourth line contains a single integer P representing the number of soldiers per team. The next N lines contain M characters each representing the grid. The grid will contain exactly 3 R bases and exactly 3 B bases. All soldiers for each team start at their team's **first spawn base** (first R or B encountered in reading order).

Bohr continued…

Output:
Output two lines:
Red: X
Blue: Y
Where X and Y are the total points earned by each team over the course of the game.

Sample input:
```
10 20 10
11111 22222
33333 44444
5
..................
.R..............B.
..................
......3.......3.....
..................
..................
......3.......3.....
..................
.R............B.B.
..................
```

Sample output:
```
Red: 9
Blue: 15
```

9. Tristan

Program Name: Tristan.java Input File: tristan.dat

Tristan has become very involved in the "Sorting Club", and wants your help with a new kind of sorting he is working on. When given an array of strings, find out the minimum number of string reversals required to make the array sorted. You can reverse each string, but you cannot move the strings within the array. Each string has a reversal cost, the amount it costs to reverse said string. Find the minimum cost to make the array sorted, using the rules given.

Input:
The input will begin with one integer, `n` (`0 < n <= 1000`). Each test case will consist of two lines, the first containing m strings separated by spaces, the second containing m integers separated by spaces, each denoting the reversal cost of the corresponding string (`0 < m <= 10000`).

Output:
For each test case, output the minimum cost to sort the array of strings using reversals. If that is not possible, output `-1`.

Sample input:
```
3
aa ba ac
1 3 1
aa ab cb ac cb
1 2 1 2 1
cba bca aaa
1 2 3
```

Sample output:
```
1
3
-1
```

10. Monoco

Program Name: Monoco.java **Input File: monoco.dat**

Monoco is a bit of an odd fellow, especially with how he likes to represent different data structures. Most people would represent Binary Trees using a pointer-based representation, where each node has two pointers: one to its left child, and one to its right. However, Monoco thinks of himself as that of a connoisseur and believes that pointers are disgusting. He's been provided a series of vile, repulsive pointer-based binary trees, and wishes to convert them to the far-superior array-based representations.

Consider a pointer-based binary tree with V_{pointer} nodes. It can be shown that an array does not need more than

$$V_{\text{array}} = 2^{h+1} - 1$$

elements to properly represent the relationship between each parent and its respective children, where h is the height of the tree. It can also be shown that a parent at index i has left child at index $2i + 1$ and right child at index $2i + 2$.

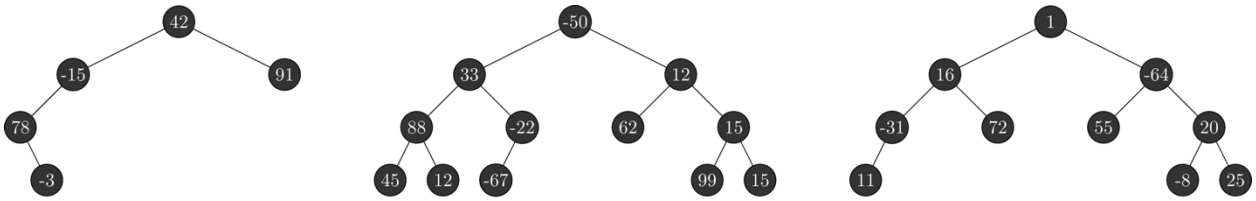
Input:
The first line will consist of a single integer T, $1 \leq T \leq 10^2$, denoting the number of testcases to follow. Each of the following T lines will consist of a single string denoting the serialized object representation of a pointer-based binary tree. Each tree is guaranteed to have $1 \leq V_{\text{pointer}} \leq 10^4$, where V_{pointer} denotes the number of vertices contained within the binary tree of the serialized object. Each node within the binary tree is guaranteed to store an integer. It is also guaranteed that $0 \leq h \leq 17$ holds for all test cases.

Output:
For each test case, on its own line, print the V_{array} integers each separated by a single space. For indices where no such value exists, instead print "`null`" instead of an integer.

Sample input: (*lines truncated at …*)
```
3
rO0ABXNyAAtCaW5UcmVlTm9kZQAAAAAAAAABAgADSQAFdmFsdWVMAARsZWZ0dAANTEJpblRyZWVOb2R...
rO0ABXNyAAtCaW5UcmVlTm9kZQAAAAAAAAABAgADSQAFdmFsdWVMAARsZWZ0dAANTEJpblRyZWVOb2R...
rO0ABXNyAAtCaW5UcmVlTm9kZQAAAAAAAAABAgADSQAFdmFsdWVMAARsZWZ0dAANTEJpblRyZWVOb2R...
```

Sample output:
```
42 -15 91 78 null null null null -3 null null null null null null
-50 33 12 88 -22 62 15 45 12 -67 null null null 99 15
1 16 -64 -31 72 55 20 11 null null null null null -8 25
```

Explanation:
The following diagram shows the logical representation of the three binary trees from the sample input/output.



Test Case #1 Test Case #2 Test Case #3

Important Notes:
There are many factors that affect how object serialization works that don't pertain to what the object logically represents. Because of this, you should avoid changing the names of existing methods/fields, creating additional `public` / `protected` methods (`private` is fine) and avoid creating any new fields as this affects what data is associated with the encoding of the object. You may set the `toString()` method to simply return the empty string if you do not wish to make use of it in your implementation.

Monoco continued…

Starter Class:

```
import java.io.Serializable;

class BinTreeNode implements Serializable {
    public static final long serialVersionUID = 1L;

    private int value;
    private BinTreeNode left, right;

    public BinTreeNode(int value) {
        this.value = value;
    }

    public Integer[] getArrayRepresentation() {
        // TODO: Implement
    }

    @Override
    public String toString() {
        // TODO: Implement
    }
}
```

Serialization Code:

```
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.util.Base64;

class Serializer {
    public static String marshall(Serializable o) throws IOException {
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(baos);
        oos.writeObject(o);
        String encodedData = Base64.getEncoder().encodeToString(baos.toByteArray());
        baos.close();
        oos.close();
        return encodedData;
    }

    public static Object unmarshall(String s) throws ClassNotFoundException, IOException {
        byte[] data = Base64.getDecoder().decode(s);
        ByteArrayInputStream bais = new ByteArrayInputStream(data);
        ObjectInputStream ois = new ObjectInputStream(bais);
        Object o  = ois.readObject();
        bais.close();
        ois.close();
        return o;
    }
}
```

Monoco continued…

Sample Usage:

```
String byteStream = file.readLine();
Object o = null;
try {
    o = Serializer.unmarshall(byteStream);
} catch (Exception e) {
    System.out.println("Come on problem setter, get your classes straight!");
    return;
}

BinTreeNode binTree = null;
if (o instanceof BinTreeNode) {
    binTree = BinTreeNode.class.cast(o);
} else {
    System.out.println("Come on problem setter, get your classes straight!");
    return;
}
```

11. Esquie

Program Name: Esquie.java **Input File: esquie.dat**

Esquie has come up with a brilliant new way of measuring time. He realized when looking at a clock that anyone who understands how many degrees are in a circle and knows some basic division can easily figure out what angle each of the hands of a clock is located at based on nothing other than the traditional `HH:MM` time format.

Let α be the clockwise angle that the hour hand of a clock sits, relative to pointing straight up ($\texttt{HH} \equiv 12$). Similarly, let β be the clockwise angle starting from the hour hand and ending at the minute hand. Then, every combination of hours and minutes can instead be modeled by the value "α: β". Esquie is not concerned about factoring in AM and PM time, since most analog clocks are unable to do the same.

Despite Esquie's genius, he is annoyingly lazy and does not wish to do these calculations by hand. Help Esquie automatically convert the traditional `HH:MM` format of time-telling to his new-and-improved α: β format.

Input:
The first line will consist of a single integer T, $1 \leq T \leq 2.5 \cdot 10^6$, denoting the number of testcases to follow. Each of the following T lines will consist of a single string denoting the time that Esquie wishes to convert, which will be in the format of `HH:MM`, where `HH` is a two-digit number denoting the hour, and `MM` is a two-digit number denoting the minute of the hour. It is guaranteed that $01 \leq \texttt{HH} \leq 12$, and $00 \leq \texttt{MM} \leq 59$ hold for all test cases.

Output:
For each test case, on its own line, print the value α: β, where both α and β are five characters wide, expressed to a single decimal place of precision.

Sample input:
```
3
9:45
7:25
12:00
```

Sample output:
```
292.5:337.5
222.5:287.5
000.0:000.0
```

Explanation:
The following analog clocks correspond to each of the three test cases above.

12. Sophie

Program Name: Sophie.java **Input File: sophie.dat**

Sophie recently got into a board game known as "Ticket to Ride", in which each player has a train color, and by obtaining different train cards, can complete their routes they are asked to complete. The board can effectively be abstracted out to a set of vertices (cities) and edges (a series of trains owned by a single player). A player is only allowed to place their colored trains along an edge between two cities so long as (1) there is an unclaimed edge between the two cities, and (2) they possess enough colored train cards corresponding to the number of slots for the unclaimed route. Once a player has claimed a specific edge, they are the only one allowed to use that edge towards completing their routes. That said, a pair of adjacent cities may have multiple unique edges that directly connect the two cities, thus multiple players may have *an edge* that directly connects two cities. For non-adjacent cities, a player is said to connect them together if any only if there exists some set of contiguous edges between the two cities that the player owns. Given a board state and a set of route cards for each player, help Sophie automatically determine whether that player has completed said route.

Input:
The first line will consist of a single integer T, $1 \leq T \leq 10^3$, denoting the number of testcases to follow. Each of the following T test cases will begin with a line of four single-space separated integers. The first of these integers is P, $2 \leq P \leq 5$ denoting the number of players for this game. The second and third will denote V and E, $2 \leq V \leq 100$, $1 \leq E \leq \frac{(V-1)(V)}{2}$, denoting the number of cities and edges between cities. The fourth will denote Q, $1 \leq Q \leq 10^3$ denoting the number of queries Sophie has regarding whether a player has completed a given route.

For each test case, the next line will consist of P single-space separated strings denoting the name of the color of each player's trains. The next line will consist of V single-space separated strings denoting the name of the V cities. The next E lines will consist of three single-space separated strings in the format of `City1 City2 Color`. This denotes that the player with color `Color` has connected `City1` and `City2` together (and vice versa). The last Q lines will consist of three single-space separated strings also in the format of `City1 City2 Color`. This denotes that Sophie wishes to know if the player with color `Color` has connected `City1` and `City2` together.

Output:
For each query, on its own line, either print the string "Route Completed" or "Route Not Completed" depending on whether the given query came back positively or negatively.

Sample input: ***(indented lines are a continuation of the previous line)***

```
1
2 48 29 8
Red Yellow
Berlin Athina Kyiv Zurich Smolensk Paris Brest Wien Angora Budapest Sofia
       Frankfurt Kobenhavn Rostov Erzurum Smyrna Petrograd Brindisi Zagrab
       Marseille London Edinburgh Amsterdam Roma Palermo Constantinople Madrid
       Barcelona Bruxelles Venezia Essen Bucuresti Danzig Wilno Stockholm Moskva
       Warszawa Pamplona Sarajevo Sevastopol Dieppe Munchen Rostov Sochi Kharkov
       Riga Cadiz Lisboa
Lisboa Cadiz Yellow
Cadiz Madrid Yellow
Lisboa Madrid Yellow
Madrid Pamplona Yellow
Madrid Barcelona Yellow
Barcelona Pamplona Yellow
Pamplona Paris Yellow
Paris Frankfurt Yellow
Frankfurt Munchen Yellow
Frankfurt Berlin Yellow
Berlin Danzig Yellow
Danzig Riga Yellow
Riga Petrograd Yellow
Zagrab Sarajevo Yellow
Sarajevo Athina Yellow
Madrid Pamplona Red
Pamplona Paris Red
Paris Bruxelles Red
Dieppe London Red
Bruxelles Amsterdam Red
Amsterdam Essen Red
Essen Berlin Red
Essen Kobenhavn Red
Kobenhavn Stockholm Red
Berlin Wien Red
Wien Budapest Red
Budapest Kyiv Red
Kyiv Warszawa Red
Riga Wilno Red
Madrid Berlin Red
Berlin Madrid Yellow
Barcelona Athina Red
Barcelona Athina Yellow
Paris Lisboa Yellow
Dieppe Berlin Red
Stockholm Wilno Red
Petrograd Munchen Yellow
```

Sample output:

```
Route Completed
Route Completed
Route Not Completed
Route Not Completed
Route Completed
Route Not Completed
Route Not Completed
Route Completed
```